

Backend Engineer (PHP Laravel)

Technical Assessment

Role: Backend Engineer (Laravel)

Duration: 96 hours (quality over speed) - Candidates are expected to **submit the completed Technical Assessment on or before Tuesday, 27 January 2026, by 5:00 PM (WAT).**

1. Objective of This Assessment

This assessment is designed to **deeply evaluate real-world backend engineering capability**, not just Laravel syntax.

We are testing your ability to:

- Design clean, scalable backend architecture
- Build secure, production-grade APIs
- Properly dockerise and deploy applications
- Model robust databases
- Write meaningful automated tests
- Demonstrate DevOps and deployment competence

This test intentionally mirrors how we build and operate production systems.

2. Business Scenario

You are building a **Digital Marketplace API** where creators sell digital products (e-books, videos, files, licenses, etc.).

The platform supports:

- Creators uploading and managing products
- Customers purchasing digital products
- Secure access to purchased files
- Scalable architecture suitable for growth

3. Mandatory Tech Stack

Your solution **must** use the following:

- **Language / Framework:** PHP 8.2+ / Laravel 10 or 11
- **Database:** PostgreSQL
- **Cache & Queues:** Redis
- **Authentication:** Laravel Sanctum
- **Containerisation:** Docker + Docker Compose
- **Web Server:** Nginx

Optional but rewarded:

- Laravel Horizon
- OpenAPI / Swagger

- Event-driven architecture

4. Dockerisation & Environment (Critical)

Your entire application must run with one command:

`docker-compose up -d`

Required Containers

- PHP-FPM
- Nginx
- PostgreSQL
- Redis

Docker Expectations

- Clean Dockerfile (multi-stage build preferred)
- Environment variables via `.env`
- Automatic:
 - Migrations
 - Seeders
- No hard-coded secrets

If the project cannot boot cleanly using Docker, the submission fails.

5. Core Functional Requirements

5.1 Authentication & Roles

- Users can register as:
 - **Creator**
 - **Customer**
- Authentication via **Sanctum**
- Role-based access control enforced at API level

5.2 Product Management (Creator)

Creators can:

- Create digital products
- Upload a digital file (PDF, ZIP, MP4, etc.)
- Update product metadata
- Replace uploaded files
- Soft-delete products

Key Rules:

- Only product owners can edit/delete
- Files must not be publicly accessible

5.3 Product Discovery (Public)

Public endpoints:

- List products
- Filter by category
- Sort by price
- View product details

5.4 Purchase & Library (Customer)

Customers can:

- Checkout a product (simulate payment)
- View purchased products (library)
- Download purchased files

Security Requirement:

- File downloads must use **temporary signed URLs**
- Only buyers can download
- Signed URLs must expire

6. API Design Requirements

- RESTful design
- Proper HTTP status codes
- Laravel API Resources for responses
- Pagination where applicable

Bonus points:

- Idempotent checkout endpoint
- API versioning strategy

7. Database Design Expectations

You are expected to:

- Design normalized tables
- Use foreign keys & indexes
- Handle:
 - Users
 - Roles
 - Products
 - Categories
 - Purchases
 - File storage references

We will evaluate:

- Schema clarity
- Relationship correctness
- Query efficiency

8. Architecture & Code Quality

Strongly Enforced

- Thin controllers
- Business logic in:
 - Service classes
 - Action classes
- Validation via Form Requests
- Clean folder structure

We Are Explicitly Looking For

- SOLID principles
- Dependency injection
- Meaningful naming
- Readable, maintainable code

9. Queues, Events & Background Jobs

At least one of the following must be implemented using queues:

- Purchase processing
- Email notification (mock)
- File access logging

Redis **must** be used.

10. Testing (Very Important)

Mandatory Tests

- Feature tests for:
 - Authentication
 - Checkout flow
 - Download authorization

Expectations

- PHPUnit / Pest
- Use database transactions or refresh
- Clear test naming

Low-quality or missing tests will significantly reduce score.

11. API Documentation (Mandatory)

Provide **one** of the following:

- Swagger / OpenAPI (preferred)
- Postman collection (postman_collection.json)

Documentation must allow us to test immediately.

12. Deployment & DevOps Skills (Critical)

You must include:

12.1 Production Deployment Guide

In your README:

- How you would deploy this to:
 - AWS / DigitalOcean / GCP
- Environment setup
- Queue workers
- Storage configuration

12.2 CI/CD (Conceptual)

Provide either:

- GitHub Actions YAML **or**
- Written explanation of:
 - Build
 - Test
 - Deploy pipeline

13. Seed Data (Mandatory)

Seeders must create:

- At least 1 Creator account
- At least 1 Customer account
- Sample products

Credentials must be documented in README.

14. Submission Requirements

Submit:

1. GitHub or GitLab repository
2. README.md containing:
 - Setup instructions
 - Architecture explanation
 - Security decisions
 - Deployment notes
3. Clear commit history

15. Evaluation Criteria

Area	Weight
Architecture & Code Quality	25%
Docker & Environment	20%
Database Design	15%

Security & Authorization	15%
Testing	15%
Deployment & DevOps	10%

16. Automatic Disqualifiers

- No Docker support
- Broken setup
- Hard-coded secrets
- No tests
- Poor security around downloads

17. What Will Impress Us

- Clean, opinionated architecture
- Thoughtful trade-offs
- Production mindset
- Clear documentation
- Defensive coding

This is not a tutorial test.

This is a backend engineer evaluation.

Good luck.